Listing 3.4 shows the implementation of the internal method taking a node in input and traversing its subtree; this version uses recursion (a technique described in appendix E): it's worth repeating that, although recursion often results in cleaner code when applied to iterative data structures like trees, recursive methods can cause stack overflow, if the depth of the recursion is significant. In this particular case, most programming languages' compiler will be able to apply tail call optimization and transform recursion into an explicit loop, while translating the code into machine language[4]; generally, however, it might be worth considering writing directly the explicit loop equivalent even in the higher level language, especially if you are not sure about your compiler support for tail recursion optimization, or the conditions where it can be applied.

Listing 3.4 method search

```
function search(node, targetKey)
                                      #1
  if node == null then
                  #2
    return null
  if node.key == targetKey then
                           #3
    return node
  elsif targetKey < node.key then
                  #4
    return search(node.left, targetKey)
                  #5
  else
    return search(node.left,
targetKey)                                          #8
```

#1 Method `search` takes a treap's node and the key to search; it returns the node holding the key, if found, otherwise `null`.